



Universität Regensburg

Faculty of
Informatics and Data Science
Department of
Statistical Bioinformatics

Bachelor Thesis

Ben J. W. Weber

Implementing Self-Distillation with No Labels for Lymphoma Biopsies in Julia

Matriculation Number	2355270
Supervisor	Prof. Dr. Rainer Spang
Submission Date	30th September 2026

Eidesstattliche Erklärung

Ich habe die Arbeit selbstständig verfasst, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt und bisher keiner anderen Prüfungsbehörde vorgelegt. Außerdem bestätige ich hiermit, dass die vorgelegten Druckexemplare und die vorgelegte elektronische Version der Arbeit identisch sind, dass ich über wissenschaftlich korrektes Arbeiten und Zitieren aufgeklärt wurde und dass ich von den in § 27 Abs. 5 vorgesehenen Rechtsfolgen Kenntnis habe.

Regensburg, den 30. September 2026

Gezeichnet

Acknowledgements

I would like to thank Lukas Wolfseher for the guidance on this thesis.

Abstract

Automated cancer sub-typing from lymphoma biopsy whole slide images (WSIs) could accelerate the diagnostic process and enable more educated treatment decisions. WSIs are commonly divided into individual tiles to make them compatible with standard computer vision pipelines. Those local tiles, however, do not necessarily correspond well to slide-level labels. This makes training traditional supervised classifiers challenging. Self-supervised approaches have shown success in learning superior image features without relying on labels. DINOv2, a method for training self-supervised vision foundation models, trains a student network alongside a mean teacher network to arrive at meaningful image representations. This approach has been successfully applied to a broad dataset of WSI tiles from diverse origins. Lymphoma biopsy WSIs might benefit from a model trained on more specific data. In this thesis, we implement DINOv3, a revision of DINOv2, in Lux.jl, a pure Julia deep-learning framework. We adapt the existing method to run on fewer devices with smaller batch sizes. We propose continued pre-training from available DINOv3 checkpoints trained on general web images (LVD-1689M). Initializing the student and teacher backbones from existing parameters allows us to leverage the training already performed for DINOv3. We were unable to evaluate our approach as training aborted due to a memory leak that accumulates over iterations. Analysis shows that the presented implementation suffers from increased compilation overhead. Profiling suggests that later iterations run at a constant pace. The implementation is available at <https://github.com/benjwweber/BachelorThesis>.

Table of Contents

1 Introduction	1
2 Background	3
2.1 Vision Transformer	3
2.2 Self-Distillation with No Labels	5
2.2.1 Mean Teacher Knowledge Distillation	5
2.2.2 Image-Level Objective	5
2.2.3 Patch-Level Objective	6
2.2.4 KoLeo Regularization	7
2.2.5 Gram Anchoring	7
2.2.6 Knowledge Distillation	8
2.3 Lux.jl and Reactant.jl	9
3 Methods	10
3.1 Continued Pre-Training	10
3.2 Adaptation to Small Batch Sizes	11
3.3 Adaptation to Low Sharpening Temperatures	11
3.4 Training Details	12
4 Results	13
4.1 Training Attempts	13
4.2 Profiling the Training Process	13
5 Discussion & Conclusion	15
Bibliography	16
Appendix	a
A Model Architecture	a
A.1 Vision Transformer	a
A.2 Projection Heads with Bottleneck	d
B Implementation Details	e
C Data Augmentation	e
D Reproducibility	g
E Additional Implementations	h
E.1 Attention-based Multiple-Instance Learning	h

List of Figures

Figure 1: Image to Sequence The ViT turns an input image into a sequence of embeddings. The input is of size $W \times H \times C$. Patches of $K \times K$ non-overlapping pixels are considered one token. The tokens are flattened into a sequence and P extra tokens are prefixed, one [CLS] and $P - 1$ [REG] tokens. The final sequence has $P + W' \times H'$ embeddings of dimension D , with $W' = \lfloor W/K \rfloor$ and $H' = \lfloor H/K \rfloor$. The [REG] tokens are later disregarded. 3

Figure 2: Transformer Block Each transformer block consists of a MHSA layer and a FFN. Each layer is preceded by a norm layer and followed by a scale layer. Those three layers are enclosed in a residual connection each. A dropout before the connection facilitates stochastic depth. L transformer blocks are chained together to build the transformer encoder. 4

Figure 3: Image-Level DINO Objective Global and local views of the same image are passed through the student network. The teacher network only receives the global views. The DINO objective considers the cross-entropy between prototype distributions of different views from different networks. The teacher network is updated via an EMA of the student network. 6

Figure 4: Patch-Level iBOT Objective Patch tokens of the student network are randomly masked. The teacher network receives all patches unmasked. The iBOT objective considers the cross-entropy between prototype distributions of the masked student patches and the corresponding teacher patches. The teacher network is updated via an EMA of the student network. 7

Figure 5: Time Profile of Training Iterations Iterations in which an optimizer step is performed (**b**) are noticeably longer than iterations that only accumulate the gradient (**a**). The difference is mainly caused by garbage collection (GC). We report a measurement representative of observations made over several iterations. Operations run on the CPU and GC are included under *others*. 14

Figure 6: Samples of the Data Augmentation Views are generated from the original tile (left). Two global views (middle) and eight local views (right) are produced. Data augmentations include flipping, color jitter, grayscale, blur, and solarization. Masks are generated for approximately half of the global views. We depict them by overlaying the corresponding patches in black. g

List of Tables

Table 1: ViT Configurations for which pre-trained weights are available Weights are available for models trained on a general web image dataset (LVD-1689M) [1] **(a)** and a satellite image dataset (SAT-493M) [1] **(b)**. The ViT-7B/16 models are the only models resulting from DINOv3 pre-training. Smaller configurations are distilled using the 7B parameter model as a fixed teacher. 8

Table 2: Profile of Compiled Functions Overhead of MLIR/XLA compilation is observed once at compile time and first Julia run time. Subsequent run times are stable and reported as a mean measurement over 16 iterations. The reported memory is the peak memory allocation during the function’s lifetime. Most time associated with the overhead is spent in the first optimizer step execution. 13

List of Algorithms

Algorithm 1: Linear Layer in Lux.jl and PyTorch A simplified implementation of a linear layer in both Lux **(a)** and PyTorch **(b)**. The PyTorch layer stores parameters in the layer object, relying on internal mutation to update. The Lux.jl layer only stores the relevant dimensions. Parameters are passed in the function signature and can be updated without modifying the model. 9

Algorithm 2: Comparison of Sinkhorn-Knopp Implementations Simplified implementations of the iterative Sinkhorn-Knopp algorithm, assuming only two-dimensional input $X \in \mathbb{R}^{K \times B}$. The reference implementation **(a)** and the algorithm adapted to log-space **(b)**. The reference implementation tends to produce NaNs for sufficiently small temperatures. Our implementation postpones the exponentiation to a point where elements are in a safer regime and uses a numerically stable logsumexp implementation mitigating the risk of introducing non-finite floats. The macro `@. f(x)` expands to the element-wise operation `f.(x)`. Functions marked by `$f(x)` are exempt. 12

1 Introduction

Whole slide images (WSIs) are digital high-resolution scans of prepared tissue slices. WSIs of lymphoma biopsies are utilized in the diagnostic process for cancer of the lymphatic system. Identifying the exact sub-type can result in more educated treatment plans. This process is generally time intensive and requires years of experience. Machine learning promises to accelerate and assist the diagnostic process through automation.

We use tiles extracted from WSIs to train vision models for this reason. Scans can contain several gigabytes of data and vary in resolution. The whole slide is therefore divided into individual tiles via a quad tree. Tiles are available at different zoom levels, with a fixed resolution of 512×512 pixels. This makes the tiles compatible with standard training and inference pipelines.

Lymphoma biopsy WSIs are usually associated and annotated with the patient’s diagnosis. However, assigning those slide-level diagnostic labels to the tile-level images is inaccurate at best. Local regions of the slide might exhibit features not consistent with the slide-level label. Some tiles may not depict cancerous tissues at all.

Learning meaningful representations from images without relying on labels has been a research focus in the computer vision domain for some time. Single labels tend to provide insufficient descriptions of images.

Self-distillation with no labels (DINO) [2] first demonstrated that vision transformers (ViTs) can immensely profit from self-supervised learning (SSL). Emerging properties like semantic segmentation information are not observed in supervised training approaches. The method is presented in terms of mean teacher self-distillation [3]. A momentum encoder [4] is paired with a multi-crop augmentation strategy [5].

DINOv2 [6] combines the image-level DINO objective with the patch-level iBOT [7] objective. Furthermore, a KoLeo [8] regularization term is added, forcing a more uniform distribution over the latent space. The softmax-centering is replaced by the iterative Sinkhorn-Knopp batch-wise normalization of SwAV [5]. DINOv2 achieves a more stable configuration, enabling pre-training to scale in terms of both data and model size.

DINOv2 has been successfully applied to WSIs of various cancers. UNI [9] introduces a general-purpose pathology model pre-trained on more than 100 million tiles sampled from over 100,000 diagnostic WSIs of major tissue types. UNI achieves state-of-the-art performance across multiple pathology tasks.

The dataset on which UNI is pre-trained contains around 5,000 slides from the lymphatic system. Only tiles at the maximum zoom level are included. All WSIs included are of the same stain [9]. Models specifically used on lymphoma biopsy WSIs might profit from more in-domain training.

In this thesis we implement the most recent evolution of the DINO method DINOv3 [1] in Lux.jl. We adapt the method to work with smaller batch sizes by accumulating gradients and

caching teacher outputs to augment Sinkhorn-Knopp normalization. This enables training on fewer devices.

We attempt to continue the pre-training process of a ViT-L/16 initialized from a publicly available checkpoint. Our dataset consists of 12,838,642 tiles from 11,292 lymphoma biopsy WSIs, including 11,147 H&E- and 145 CD20-stained slides. We include tiles at multiple zoom levels, specifically the last three levels. The pre-training has yielded no results so far.

We show that the current training runs are dominated by the compilation time. Apart from the compilation overhead, the measured iteration times suggest that training may be feasible once the memory-management issues are resolved. Finally, we discuss possible directions for future work.

2 Background

2.1 Vision Transformer

Vision transformers (ViTs) [10] have become an established model choice in various vision domains. ViTs are based on the transformer [11] architecture, originally proposed for sequence transduction in natural language processing (NLP). At its core, it utilizes the same encoder as a transformer, with only minimal modifications to accommodate the two spatial dimensions of image input [10].

DINOv3 uses an updated version of the original ViT as the backbone model. More specifically, it adds registers [1], [12] to the ViT and replaces the original absolute positional embedding (APE) with a rotary positional embedding (RoPE) [1], [13], [14].

An input image is divided into contiguous non-overlapping equal-sized patches. These patches represent the tokens the model operates on. Each patch is projected to an embedding vector, using a convolutional layer with equal kernel size and stride. The resulting embeddings are flattened into one sequence and prefixed with the embeddings of one classification ([CLS]) token and several register ([REG]) tokens [12] (see Figure 1).

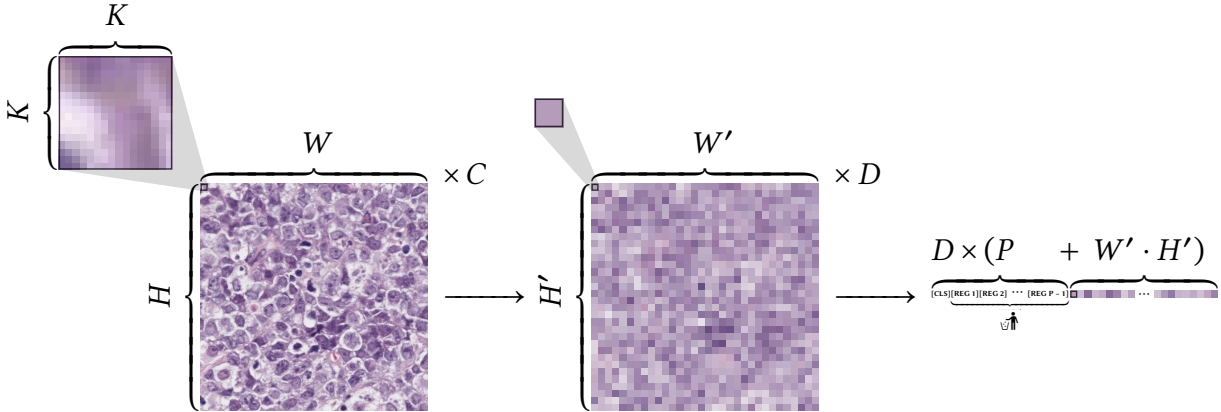


Figure 1: Image to Sequence The ViT turns an input image into a sequence of embeddings. The input is of size $W \times H \times C$. Patches of $K \times K$ non-overlapping pixels are considered one token. The tokens are flattened into a sequence and P extra tokens are prefixed, one [CLS] and $P - 1$ [REG] tokens. The final sequence has $P + W' \times H'$ embeddings of dimension D , with $W' = \lfloor W/K \rfloor$ and $H' = \lfloor H/K \rfloor$. The [REG] tokens are later disregarded.

The embeddings of these prefix tokens have the same dimension as the embeddings of the patch tokens and are learnable parameters. The [REG] tokens provide temporary space for additional global information and are later disregarded [12]. The [CLS] token’s final embedding vector is the representation of the image later used in classification tasks [10], [15]. The resulting sequence of [CLS], [REG], and patch token embeddings is passed through the encoder.

The encoder consists of a stack of alternating multi-head self-attention (MHSA) layers and feed-forward networks (FFNs), in our case simple multi-layer perceptrons (MLPs). Each of

these layers is preceded by a normalization layer and succeeded by a scale layer. A residual connection encloses all three layers (see Figure 2).

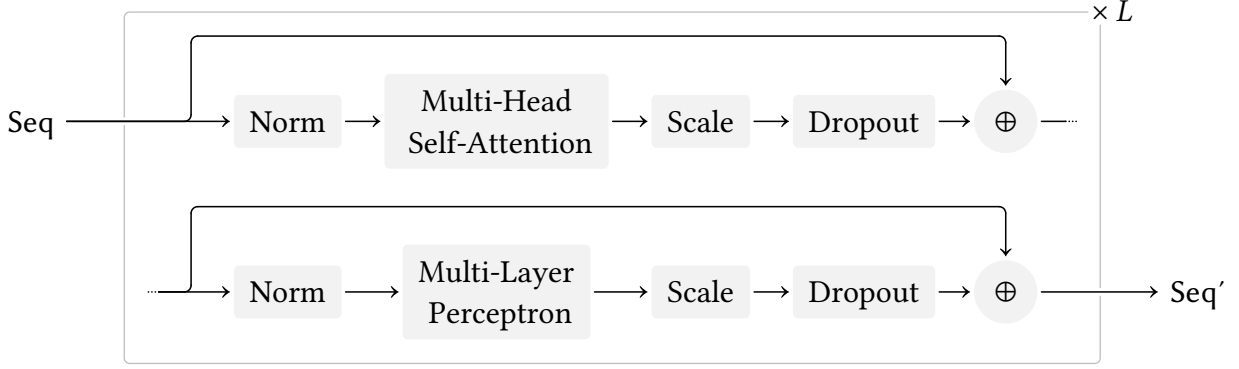


Figure 2: Transformer Block Each transformer block consists of a MHSA layer and a FFN. Each layer is preceded by a norm layer and followed by a scale layer. Those three layers are enclosed in a residual connection each. A dropout before the connection facilitates stochastic depth. L transformer blocks are chained together to build the transformer encoder.

The MLPs are applied to the sequence token-wise, acting separately and identically on each embedding [11]. The MHSA layers enable the model to exchange information across the sequence using scaled dot-product attention (SDPA) [11]. Each token can attend to every other token, itself included, by computing a weighted average over the sequence’s embeddings. The influence of one token on another is determined by the corresponding attention score $\alpha_{i,j}$,

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathbf{V} \times \mathbf{A} = \mathbf{V} \times \text{softmax}\left(\frac{\mathbf{K}^T \times \mathbf{Q}}{\sqrt{d_K}}\right).^1$$

Self-attention means that the query ($\mathbf{Q}$), the key ($\mathbf{K}$), and the value ($\mathbf{V}$) sequences are all linear projections of the same input sequence ($\mathbf{X}$)

$$\mathbf{q}_i = \mathbf{W}_Q \times \mathbf{x}_i + \mathbf{b}_Q, \quad \mathbf{k}_i = \mathbf{W}_K \times \mathbf{x}_i + \mathbf{b}_K, \quad \mathbf{v}_i = \mathbf{W}_V \times \mathbf{x}_i + \mathbf{b}_V.$$

Multi-head attention divides the query’s, key’s, and value’s embedding dimension onto several attention heads. Each head performs SDPA in parallel and the results are concatenated.

SDPA is generally unaware of the relative positional relationships between tokens. RoPE introduces this information directly into $\mathbf{Q}$ and $\mathbf{K}$. For each patch token in each sequence, pairs of features are rotated via a rotation matrix according to the token’s original position. Half of the pairs are rotated with respect to the x-position, the other half with respect to the y-position. This way each patch token receives a unique signature. For a fixed query token $\mathbf{q}$ corresponding to patch (i, j) and a key token $\mathbf{k}$ corresponding to patch (k, l) , the attention score is a function only of their relative positions $i - k$ and $j - l$.

The transformer encoder is followed by a final norm layer. The ViT returns both the [CLS] and patch tokens’ final embeddings. [REG] tokens are usually disregarded.

For a more detailed description of the underlying layers, please refer to Appendix A.1.

¹Adapted from [11] to Lux.jl’s column-major data-layout, i.e. $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{d \times N}$.

2.2 Self-Distillation with No Labels

Self-distillation with no labels version 3 (DINOv3) is the third iteration of the DINO SSL method. This section provides a short overview of the method. DINOv3 uses self-supervision to train a ViT. Training is done in two stages. In the initial stage a combination of image-level DINO (see Section 2.2.2), patch-level iBOT (see Section 2.2.3) and KoLeo regularization (see Section 2.2.4) objectives are optimized. In the second refinement stage a Gram teacher is added to ensure patch-level consistency (see Section 2.2.5).

All descriptions are derived from the original papers [1], [2], [6] and the publicly available implementations [16], [17], [18].

2.2.1 Mean Teacher Knowledge Distillation

DINOv3 is presented in terms of mean teacher self-distillation [2], [3]. A student network is trained to match the features of a teacher network. Both the student and the teacher share the same model architecture g , but differ in their parameterization θ_s and θ_t . Both parameterizations initially start from the same point. An objective J is optimized with respect to the parameters of the student

$$\min_{\theta_s} J(g_{\theta_s}(\mathbf{x}), g_{\theta_t}(\mathbf{x})).$$

Therefore only the student receives gradient updates from the optimizer. The teacher acts as a momentum encoder [4] updated via an exponential moving average (EMA) with the momentum λ

$$\theta_t \leftarrow \lambda \cdot \theta_t + (1 - \lambda) \cdot \theta_s.$$

DINOv3 uses a ViT as the backbone model and two separate heads for the DINO objective (see Section 2.2.2) and the iBOT objective (see Section 2.2.3). The final teacher backbone is used as the pre-trained model for downstream tasks.

2.2.2 Image-Level Objective

The original DINO objective is an image-level multi-view objective. It considers the cross-entropy between student and teacher class features from different views of the same image (see Figure 3).

For each observation a set of different augmented views $\{\mathbf{x}_i \mid i \in I\}$ is generated. This set contains two *global* views and several *local* views. Global views cover a large area of the original image at higher resolution. Local views are constrained to smaller areas at a lower resolution.

To produce class features the view is passed through the backbone (ViT). The resulting [CLS] token is passed through an additional DINO head (MLP). The MLP outputs scores over K prototypes [5]. The probability distribution $\mathbf{p}_{s,i}$ is obtained by softmax normalizing the student network’s prototype scores. To obtain the probability distribution $\mathbf{p}_{t,i}$ the teacher network’s

prototype scores are sharpened and normalized across the batch using an iterative Sinkhorn-Knopp algorithm [5].

Both the global and local views are passed through the student network, while the teacher network only ever sees global views (see Figure 3). The objective is computed as the sum of cross-entropy between the teacher and student prototype distributions of different views

$$\mathcal{L}_{\text{DINO}} = - \sum_{i \neq j} \mathbf{p}_{t,i} \log \mathbf{p}_{s,j}.$$

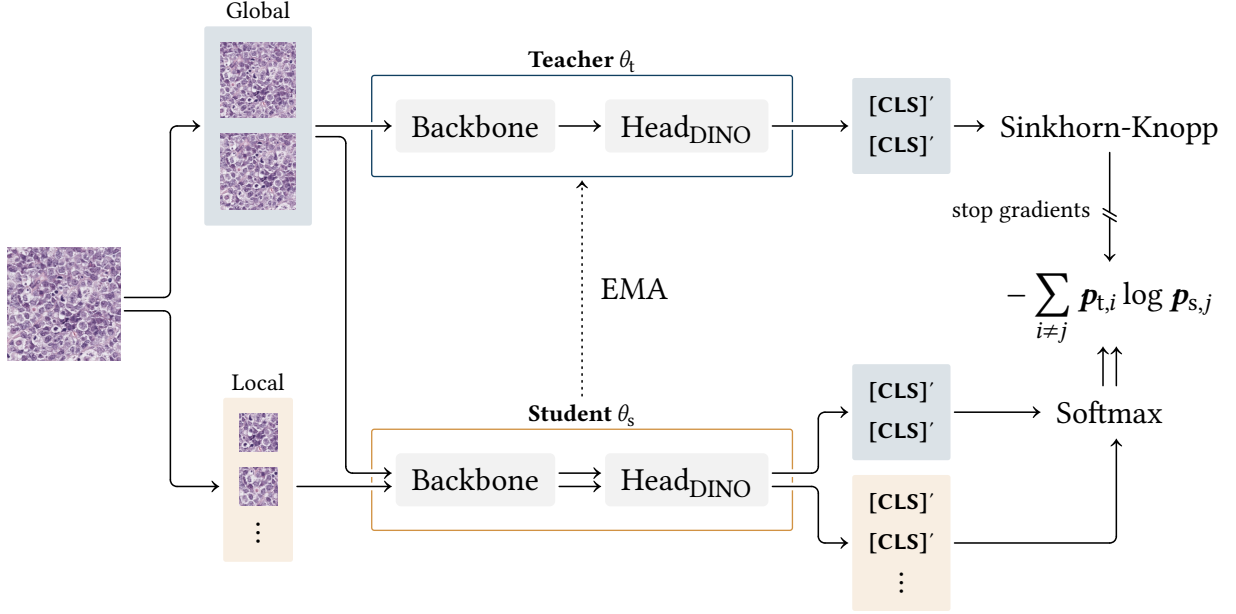


Figure 3: Image-Level DINO Objective Global and local views of the same image are passed through the student network. The teacher network only receives the global views. The DINO objective considers the cross-entropy between prototype distributions of different views from different networks. The teacher network is updated via an EMA of the student network.

2.2.3 Patch-Level Objective

DINOv2 adapts the iBOT [7] objective, a patch-level latent reconstruction objective. It does masked image modeling (MIM) [7], [15] by considering the cross-entropy between student and teacher patch features, where only the teacher sees the original patches (see Figure 4).

Features are produced by passing an image through the backbones (ViT) of the student and teacher networks. Patches $\mathbf{h}_i$ are randomly masked in the student network by substituting the patch embedding with a learnable [MASK] embedding. The teacher network is passed the full context without masked patches (see Figure 4). The resulting patch embeddings of the masked patches and the corresponding unmasked patch embeddings of the teacher are run through an additional iBOT head (MLP). Probability distributions $\mathbf{p}_{t,i}$ and $\mathbf{p}_{s,i}$ are produced as described in Section 2.2.2. The index corresponds to the i -th masked patch in this case and not the view.

The iBOT objective is computed as the cross-entropy of the reconstructed patch embeddings of the student and the corresponding embeddings of the teacher

$$\mathcal{L}_{\text{iBOT}} = - \sum_i \mathbf{p}_{t,i} \log \mathbf{p}_{s,i}.$$

The iBOT objective reuses the global views passed through the student and teacher network by the DINO objective.

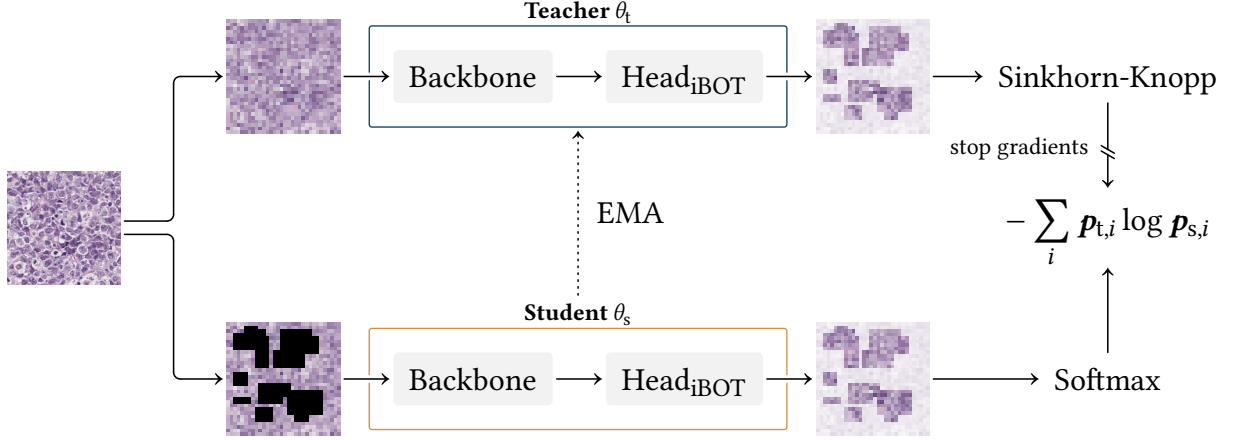


Figure 4: Patch-Level iBOT Objective Patch tokens of the student network are randomly masked. The teacher network receives all patches unmasked. The iBOT objective considers the cross-entropy between prototype distributions of the masked student patches and the corresponding teacher patches. The teacher network is updated via an EMA of the student network.

2.2.4 KoLeo Regularization

DINOv2 adds a further Kozachenko-Leonenko (KoLeo) regularization [8] loss term. It encourages uniform coverage of the latent space by penalizing the intra-batch nearest neighbor distances.

Let $\{\mathbf{y}_i \mid i \in \{1, \dots, B\}\}$ be the L_2 -normalized [CLS] token of one batch. We denote the distance between an output $\mathbf{y}_i$ and the corresponding nearest neighbor in the batch of B with $d_{B,i} = \min_{j \neq i} \|\mathbf{y}_i - \mathbf{y}_j\|_2$. The KoLeo regularization penalty is computed as

$$\mathcal{L}_{\text{KoLeo}} = -\frac{1}{B} \cdot \sum_i \log d_{B,i}.$$

DINOv3 uses a distributed implementation in which the KoLeo regularization is applied in small batches of 16 observations [1].

2.2.5 Gram Anchoring

Gram anchoring is introduced in DINOv3 to mitigate the degradation of patch-level consistency during extended periods of training. An earlier teacher parameterization with superior patch-level consistency is used to push the student’s features into the target structure. Operating on the Gram matrix pushes the patch features towards the structure of the earlier model

without restricting the features directly. The earlier model is referred to as the Gram teacher and distinct from the momentum encoder.

Let Y_s be the L_2 -normalized local patch features of the student and Y_g those of the Gram teacher. The loss is computed as

$$\mathcal{L}_{\text{Gram}} = \|Y_s^T Y_s - Y_g^T Y_g\|_F^2.$$

The Gram anchoring reuses the global views passed through the student network by the DINO objective. Optionally Y_g can be computed by feeding the Gram teacher the views at a higher resolution and down-sampling the patch features. This approach results in smoother and more coherent patch-level features.

After 1 million iterations, training enters the refinement stage, where the Gram teacher is initialized from the current teacher network. The Gram anchoring is added to the optimized objectives. Unlike the EMA teacher, the Gram teacher is not updated by the student network. The Gram teacher is updated every 10,000 iterations to match the current EMA teacher network.

2.2.6 Knowledge Distillation

DINOv3 pre-trains a ViT-7B/16 model. The pre-trained 7B parameter model can be distilled into smaller ViT configurations. The knowledge distillation uses the same combined objective as the initial pre-training stage (no Gram teacher). During distillation, the ViT-7B/16 model is used as a fixed teacher instead of relying on a teacher updated via EMA. DINOv3 checkpoints are publicly available (see Table 1).

a

LVD-1689M					
Name		Depth	Heads	Features	Parameters
ViT-S/16	distilled	12	6	384	21M
ViT-S+/16	distilled	12	6	384	29M
ViT-B/16	distilled	12	12	768	86M
ViT-L/16	distilled	24	16	1024	300M
ViT-H+/16	distilled	32	20	1280	840M
ViT-7B/16		40	32	4096	6,716M

b

SAT-493M					
Name		Depth	Heads	Features	Parameters
ViT-L/16	distilled	24	16	1024	300M
ViT-7B/16		40	32	4096	6,716M

Table 1: ViT Configurations for which pre-trained weights are available Weights are available for models trained on a general web image dataset (LVD-1689M) [1] **(a)** and a satellite image dataset (SAT-493M) [1] **(b)**. The ViT-7B/16 models are the only models resulting from DINOv3 pre-training. Smaller configurations are distilled using the 7B parameter model as a fixed teacher.

2.3 Lux.jl and Reactant.jl

Lux.jl [19], [20] is a deep-learning framework written entirely in Julia [21]. Unlike in PyTorch, layers do not store any parameters or states in Lux. They are immutable and act like pure functions, producing deterministic outputs on known inputs. Any parameters and states are passed alongside the input. A new set of possibly modified states is returned alongside the outputs (see Algorithm 1a). Stochasticity is managed by passing a random number generator (RNG) via the states and returning a stepped RNG.

Lux.jl runs on the CPU, GPU² and TPU using Reactant.jl [22] as a backend. Reactant.jl compiles a Julia function down to MLIR [23], making it available for advanced optimization, and creates the relevant executables via XLA [24]. These optimizations include using Enzyme [25], [26] for MLIR/LLVM-level automatic differentiation (AD) and Shardy [27] for sharding and cross-device communication.

a	b
<pre>using Lux struct Linear <: Lux.AbstractLuxLayer in_dims::Int out_dims::Int end function Lux.initialparameters(rng, l::Linear) weight = rand(rng, l.out_dims, l.in_dims) bias = rand(rng, l.out_dims) return (; weight, bias) end function (::Linear)(x, parameters, states) y = parameters.weight * x .+ parameters.bias return y, states end</pre>	<pre>import torch class Linear(torch.nn.Module): def __init__(self, in_dims, out_dims): super().__init__() self.weight = torch.nn.Parameter(torch.rand(out_dims, in_dims)) self.bias = torch.nn.Parameter(torch.rand(out_dims)) return None def forward(self, x): y = x @ self.weight.T + self.bias return y</pre>

Algorithm 1: Linear Layer in Lux.jl and PyTorch A simplified implementation of a linear layer in both Lux (a) and PyTorch (b). The PyTorch layer stores parameters in the layer object, relying on internal mutation to update. The Lux.jl layer only stores the relevant dimensions. Parameters are passed in the function signature and can be updated without modifying the model.

²Native GPU support is possible through the CUDA.jl and cuDNN.jl (NVIDIA), AMDGPU.jl (AMD), Metal.jl (Apple), and oneAPI.jl (Intel) packages respectively. In almost all cases Reactant.jl is the preferred backend (please refer to the official documentation).

3 Methods

The DINOv3 and ViT implementations in this thesis are based on the DINOv3 paper [1] and the official reference PyTorch implementation and models for DINOv3 publicly available on GitHub [18]. The code of this project is publicly available at: <https://github.com/benjwweber/BachelorThesis>.

The Lux.jl implementation developed in this thesis aims to stay faithful to the reference implementation, while maintaining idiomatic Julia and Lux.jl code. Known deviations are documented in Appendix B. We use Lux.jl’s built-in layers, provided abstractions, and public APIs when possible. We implemented all necessary parts of DINOv3 including the ViT, the DINO and iBOT objectives, KoLeo regularization, and Gram anchoring. Gram anchoring is not currently utilized.

3.1 Continued Pre-Training

Meta released DINOv3 model weights for several common backbone configurations [18] (see Table 1). Models are pre-trained on either general web images (LVD-1689M) [1] or satellite images (SAT-493M) [1]. Weights are available for the original ViT-7B/16 models used in large-scale pre-training and for smaller model configurations distilled from the 7B parameter model.

Instead of randomly initializing the models, we use the available checkpoints. We continue the pre-training by initializing our student and teacher networks from the corresponding checkpoint. The publicly available checkpoints only contain the backbone weights. Both training heads are therefore randomly initialized. Student and teacher receive the same initialization.

We schedule the continued pre-training in two phases. In both phases we optimize a combination of the DINO and iBOT objectives and KoLeo regularization [1], [6]

$$\mathcal{L}_{\text{Pre}} = \mathcal{L}_{\text{DINO}} + \mathcal{L}_{\text{iBOT}} + 0.1 \cdot \mathcal{L}_{\text{KoLeo}}.$$

In the first phase we aim to adapt the randomly initialized heads to the pre-trained backbone [28]. The backbone, except for the patch embedding layer, is frozen. This allows the heads to align to the backbone. At the same time, this prevents disruptively large gradients from the randomly initialized heads from updating the pre-trained transformer encoder during early training steps. Leaving the patch embeddings unfrozen allows the model to adapt to the pixel distribution of the lymphoma biopsy tiles [28]. We schedule a linear warm-up over the first 3,000 steps. To prevent early representation collapse, we also freeze the last projection layer of the DINO and iBOT head for the first 450 steps [1], [6], [28]. After aligning the DINO and iBOT heads, we unfreeze the pre-trained backbone with a linear warm-up over 1,000 steps [28]. The second phase is equivalent to the initial DINOv3 phase [1].

3.2 Adaptation to Small Batch Sizes

If the number of observations in one batch is small compared to the number of prototypes $B \ll K$, Sinkhorn-Knopp forces the prototype probabilities towards a uniform distribution. To combat this when working with small batch sizes or few GPUs, we implement a solution inspired by the original SwAV [5]. We augment the current batch with the features of previous batches for the iterative Sinkhorn-Knopp algorithm. Only the current batch is considered in the loss computation. Prototypes are cached for both the DINO and iBOT head. The cache is prefilled by the teacher features before the training run.

In addition, we accumulate the gradient over a fixed number of batches to increase the effective batch size. The accumulated gradient is clipped once immediately before the optimizer step.

3.3 Adaptation to Low Sharpening Temperatures

Sufficiently low sharpening temperatures can introduce NaNs in the reference implementation of the iterative Sinkhorn-Knopp algorithm (see Algorithm 2a). For `Float32`, $\exp(x)$ overflows to `Inf32` when x exceeds `88.72284f0`, since `isfinite(exp(88.72284f0))` is false. This can result in undefined operations introducing NaN values. The resulting NaN values propagate through the summations to all elements.

To mitigate the risk of introducing non-finite floats and facilitate more aggressive sharpening, the algorithm can be translated into log-space (see Algorithm 2b). Postponing the exponentiation ensures that input values in a numerically safe regime. Divisions are replaced with subtractions and summation can be substituted by `logsumexp` implemented in a numerically stable way,

$$\text{logsumexp}(X) = \log\left(\sum_{i,j} e^{x_{i,j}}\right) = \max X + \log\left(\sum_{i,j} e^{x_{i,j} - \max X}\right).$$

The log-space implementation (Algorithm 2b) is a direct substitute for Algorithm 2a.

a

```

function sinkhornknopp(X, t; iterations=3)
    Q = @. exp(X / t)      # Inf32 hazard
    @. Q = Q / $sum(Q)      # NaN32 hazard
    K, B = size(X)
    for _ in 1:iterations
        @. Q = Q / $sum(Q; dims=2) / K
        @. Q = Q / $sum(Q; dims=1) / B
    end
    return @. Q * B
end

```

b

```

function sinkhornknopp(X, t; iterations=3)
    Q̃ = @. X / t            # log.(Q)
    @. Q̃ = Q̃ - $logsumexp(Q̃)
    K̃, B̃ = log.(size(X))
    for _ in 1:iterations
        @. Q̃ = Q̃ - $logsumexp(Q̃; dims=2) - K̃
        @. Q̃ = Q̃ - $logsumexp(Q̃; dims=1) - B̃
    end
    return @. exp(Q̃ + B̃)
end

```

Algorithm 2: Comparison of Sinkhorn-Knopp Implementations Simplified implementations of the iterative Sinkhorn-Knopp algorithm, assuming only two-dimensional input $X \in \mathbb{R}^{K \times B}$. The reference implementation (a) and the algorithm adapted to log-space (b). The reference implementation tends to produce NaNs for sufficiently small temperatures. Our implementation postpones the exponentiation to a point where elements are in a safer regime and uses a numerically stable `logsumexp` implementation mitigating the risk of introducing non-finite floats. The macro `@. f(x)` expands to the element-wise operation `f.(x)`. Functions marked by `$f(x)` are exempt.

3.4 Training Details

Training was conducted using a ViT-L/16 with four registers initialized from the public LVD-1689M checkpoint. The experiments were conducted on four NVIDIA A100-SXM4-80GB, with a Reactant.jl backend and Enzyme.jl facilitating AD. Parameters were replicated across all devices and inputs were sharded along the batch dimension. We used a batch size of 16 observations per GPU. Each observation consisted of 2 global views at a resolution of 256×256 and 8 local views at a resolution of 128×128 . Hyperparameters were generally adopted from the ImageNet-1k [29] configurations [18]. We reduced the number of prototypes of both DINO and iBOT heads to $K = 4096$. Teacher outputs from the previous 15 batches were cached to augment the iterative Sinkhorn-Knopp algorithm. Gradients were accumulated over 16 iterations, increasing the effective batch size to 1024 observations per step. We aligned the heads for 10,000 steps [28] with warm-ups as described in Section 3.1. AdamW [30] was used as the optimizer [1]. Tile loading, view augmentation and batching run asynchronously on 16 distributed workers. Host-to-device transfer runs in parallel with the training iteration. The next batch is pre-loaded to the device while the current iteration takes place.

4 Results

4.1 Training Attempts

The presented implementation failed to complete pre-training. The pre-training aborted after approximately 60–90 optimizer steps (1000–1500 iterations). Training aborted before the backbone (ViT) was unfrozen. The training loop was observed to leak up to 500 MB of non-reclaimable memory per iteration. The program halted in one of several ways.

- The training loop aborted with an out-of-memory error.
- The training loop aborted due to a `ReadOnlyMemoryError`.
- Julia exited with a segmentation fault, caused by either of the above errors.
- The program eventually exhausted the available memory (503 GB), halting the execution of the program and all other processes on the server.

Early terminations were reproducible across different per-GPU batch sizes (8, 16, 32). Investigating and fixing the underlying issues was beyond the time and scope of this thesis.

4.2 Profiling the Training Process

For short training runs the time profile was dominated by the compilation and first execution of one function. We profiled training attempts with the setup described in Section 3.4. We utilized the built-in profiling tools `Reactant.with_profiler` and analyzed the traces using `XProf`.

The overhead created by compilation and first execution took approximately 6.5 hours. The vast majority of that time (92.13%) was spent on the first execution of `apply_gradients!` (see Table 2). Excluding `apply_gradients!` reduced the overhead down to 23 minutes.

Function	Compile Time [s]	First Run [s]	Run Time [ms]	Memory [MiB]
<code>update_queues!</code>	176.12	15.69	63.26	5984.92
<code>compute_targets!</code>	19.96	0.17	32.39	6664.81
<code>compute_gradients</code>	357.13	32.18	293.73	29366.28
<code>decumulate_gradients!</code>	59.58	26.47	4.31	1216.67
<code>accumulate_gradients!</code>	195.10	74.20	6.69	2433.33
<code>clipnorm_gradients!</code>	146.21	22.01	12.63	2433.33
<code>apply_gradients!</code>	470.94	21624.87	15.45	4866.68
<code>update_teacher!</code>	180.44	70.12	7.77	3649.99

Table 2: Profile of Compiled Functions Overhead of MLIR/XLA compilation is observed once at compile time and first Julia run time. Subsequent run times are stable and reported as a mean measurement over 16 iterations. The reported memory is the peak memory allocation during the function’s lifetime. Most time associated with the overhead is spent in the first optimizer step execution.

After compilation and first execution, iterations only accumulating the gradients took approximately 500 milliseconds. Iterations in which an optimizer step occurs took approximately 750 milliseconds. We observed that most of the additional time was spent in garbage collection (see Figure 5).

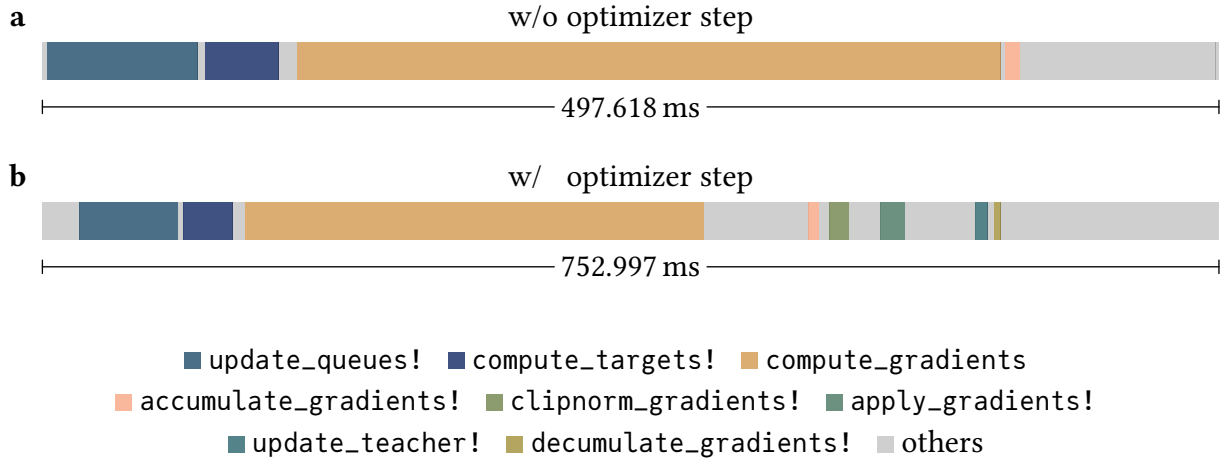


Figure 5: Time Profile of Training Iterations Iterations in which an optimizer step is performed (**b**) are noticeably longer than iterations that only accumulate the gradient (**a**). The difference is mainly caused by garbage collection (GC). We report a measurement representative of observations made over several iterations. Operations run on the CPU and GC are included under *others*.

Data-loading and host-to-device transfers were not observed to impact the training iteration run time in the profiled training attempts. Host-to-device transfers were measured at 430 milliseconds per batch happening in parallel with the training iteration.

5 Discussion & Conclusion

In this thesis we implemented DINOv3 in order to train a vision backbone model for lymphoma biopsy WSI tiles. We contribute an independent implementation in the Lux.jl deep-learning framework, adapted for training on smaller scales. We adapt the queue-based approach of SwAV to circumvent uniform collapse for small batches in the iterative Sinkhorn-Knopp algorithm. Further, we translate the algorithm into log-space to mitigate the risk of floating-point overflow. We propose a continued pre-training approach to leverage existing models. We were unable to evaluate our proposed method. Training is interrupted after too few optimizer steps to make any meaningful claims regarding the presented continued pre-training approach. The observed failures do not demonstrate fundamental limitations of the presented approach, but should be considered limitations of the current implementation. We expect that all remaining implementation issues presented in this thesis can be fixed.

In Section 3.1 we describe our training approach. DINOv3 benefits from scaling to a ViT-7B/16 pre-trained on massive amounts of specially curated image data. We do not have access to the quantity of data or compute power necessary to replicate pre-training at that scale. Instead we opted to initialize our student and teacher from a publicly available ViT-L/16 checkpoint. The parameters of this smaller model are obtained by knowledge distillation of the pre-trained ViT-7B/16. The distillation optimizes the same objective we use in our continued pre-training. Checkpoints should therefore generally be aligned to work with the objective. Approaches like fine-tuning with no labels (FINO) [28] indicate that initializing from pre-trained checkpoints can yield superior training results. Unlike FINO, we do not introduce additional objectives, but try to resume with the same objective.

In Section 4.1 we outline the circumstances under which the current implementation aborts the training attempts. Memory is leaked and accumulates until the available resources are eventually exhausted. Identifying and resolving the underlying memory-management issues should be the next priority. One temporary workaround would be to train for a limited number of optimizer steps before restarting from a checkpoint. This was deemed infeasible given the overhead associated with each new run.

In Section 4.2 we showed that compilation and first execution of the optimizer step dominated the run time of our training attempts. Densely packing all parameters corresponding to the same optimizer setup into one array could reduce the number of individual function calls reaching the MLIR/XLA pipeline. We would in turn expect a reduced time spent on the first optimizer step. The parameter interface is not restricted to nested named tuples. Existing solutions like ComponentArrays.jl should be the first option to consider.

No downstream evaluation of the proposed approach was conducted, as no model was successfully trained on domain-specific data. Future models should be evaluated using an attention-based multiple instance learning (AbMIL) model (Appendix E.1). We suggest using a classifier trained on features extracted with the public checkpoints as a baseline. A classifier trained on UNI features could serve as an in-domain comparison. We hope to observe a benefit from including different stains and tiles at multiple zoom levels. It would be interesting to see how different initializations transfer to WSI tiles.

Bibliography

- [1] O. Siméoni *et al.*, “DINOv3.” [Online]. Available: <https://arxiv.org/abs/2508.10104>
- [2] M. Caron *et al.*, “Emerging properties in self-supervised vision transformers,” in *2021 IEEE/CVF international conference on computer vision (ICCV)*, 2021, pp. 9630–9640.
- [3] A. Tarvainen and H. Valpola, “Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results,” *Advances in neural information processing systems*, vol. 30, 2017.
- [4] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *2020 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2020, pp. 9726–9735.
- [5] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, “Unsupervised learning of visual features by contrasting cluster assignments,” *Advances in neural information processing systems*, vol. 33, pp. 9912–9924, 2020.
- [6] M. Oquab *et al.*, “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024.
- [7] J. Zhou *et al.*, “iBOT: Image BERT Pre-training with Online Tokenizer,” in *International Conference on Learning Representations*, 2022.
- [8] A. Sablayrolles, M. Douze, C. Schmid, and H. Jégou, “Spreading vectors for similarity search,” in *International Conference on Learning Representations*, 2019.
- [9] R. J. Chen *et al.*, “Towards a General-Purpose Foundation Model for Computational Pathology,” *Nature Medicine*, 2024, [Online]. Available: <https://www.nature.com/articles/s41591-024-02857-3>
- [10] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *International Conference on Learning Representations*, 2021.
- [11] A. Vaswani *et al.*, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [12] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski, “Vision transformers need registers,” in *International conference on learning representations*, 2024, pp. 2632–2652.
- [13] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [14] B. Heo, S. Park, D. Han, and S. Yun, “Rotary position embedding for vision transformer,” in *European Conference on Computer Vision*, 2024, pp. 289–305.
- [15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.

- [16] Meta Research, “DINO: Self-Supervised Vision Transformers with DINO.” [Online]. Available: <https://github.com/facebookresearch/dino>
- [17] Meta Research, “DINOv2: Learning Robust Visual Features without Supervision.” [Online]. Available: <https://github.com/facebookresearch/dinov2>
- [18] Meta Research, “DINOv3: Official PyTorch Implementation and Models.” [Online]. Available: <https://github.com/facebookresearch/dinov3>
- [19] A. Pal, “Lux: Explicit Parameterization of Deep Neural Networks in Julia.” [Online]. Available: <https://doi.org/10.5281/zenodo.7808903>
- [20] A. Pal, “On Efficient Training & Inference of Neural Differential Equations,” Master’s thesis, Massachusetts Institute of Technology, 2023.
- [21] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017, doi: 10.1137/141000671.
- [22] EnzymeAD, “Reactant.jl: Optimize Julia Functions With MLIR and XLA for High-Performance Execution on CPU, GPU, TPU and more.” [Online]. Available: <https://github.com/EnzymeAD/Reactant.jl>
- [23] LLVM, “Multi-Level Intermediate Representation.” [Online]. Available: <https://github.com/llvm/llvm-project/tree/main/mlir>
- [24] OpenXLA, “XLA: A machine learning compiler for GPUs, CPUs, and ML accelerators.” [Online]. Available: <https://github.com/openxla/xla>
- [25] W. Moses and V. Churavy, “Instead of Rewriting Foreign Code for Machine Learning, Automatically Synthesize Fast Gradients,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., Curran Associates, Inc., 2020, pp. 12472–12485. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/file/9332c513ef44b682e9347822c2e457ac-Paper.pdf>
- [26] W. S. Moses *et al.*, “Reverse-Mode Automatic Differentiation and Optimization of GPU Kernels via Enzyme,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, in SC '21. St. Louis, Missouri: Association for Computing Machinery, 2021. doi: 10.1145/3458817.3476165.
- [27] OpenXLA, “Shardy: MLIR-based partitioning system.” [Online]. Available: <https://github.com/openxla/shardy>
- [28] E. Gardès *et al.*, “Who Needs Labels? Adapting Vision Foundation Models With the Metadata You Already Have.” [Online]. Available: <https://arxiv.org/abs/2606.05107>
- [29] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255. doi: 10.1109/CVPR.2009.5206848.
- [30] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *International Conference on Learning Representations*, 2019.

- [31] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, “Deep networks with stochastic depth,” in *European conference on computer vision*, 2016, pp. 646–661.
- [32] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [33] M. Ilse, J. Tomczak, and M. Welling, “Attention-based deep multiple instance learning,” in *International conference on machine learning*, 2018, pp. 2127–2136.

Appendix

A Model Architecture

This Appendix A provides a more detailed description of the models used in this thesis. For consistency with the Julia implementation, all mathematical descriptions assume a colexicographic (column-major) data layout. This may mean that the concrete formalization differ from those formulated for PyTorch implementations. As long as mathematically applicable, any batch-dimensions will be omitted. Indices not exhausting the array dimensions assume the full remaining leading dimension. For the i -th embedding in a sequence $X \in \mathbb{R}^{D \times N}$ we write $x_i \in \mathbb{R}^D$.

A.1 Vision Transformer

DINOv3 uses the latest ViT architecture as a backbone model. Section 2.1 provides a general overview of the function of the model. This section provides a more formalized description of the underlying layers. We exclude simple dropout layers. Descriptions are based on our Lux.jl implementation. The model is an adaptation of the original ViT [10] using registers [12] and RoPE [14].

A.1.1 Patch Embeddings

An input image $X \in \mathbb{R}^{W \times H \times C_{\text{in}}}$ is divided into patches, which are further projected into embeddings. Each patch embedding acts as a token passed through the transformer blocks.

This is done via a convolutional layer. The stride is kept equal to the kernel size, to ensure non-overlapping patches. To keep this implementation consistent with the reference PyTorch implementation, the layer technically performs a cross-correlation, rather than a convolution.

The cross-correlation is computed as

$$y_{k,l,j} = \left(\sum_{i=1}^{C_{\text{in}}} \sum_{m,n=1}^K w_{m,n,i,j} \cdot x_{(k-1) \cdot K + m, (l-1) \cdot K + n, i} \right) + b_j,$$

for $j \in \{1, \dots, C_{\text{out}}\}$, $k \in \{1, \dots, W'\}$, and $l \in \{1, \dots, H'\}$, with $W' = \lfloor W/K \rfloor$ and $H' = \lfloor H/K \rfloor$, with parameters $W \in \mathbb{R}^{K \times K \times C_{\text{in}} \times C_{\text{out}}}$ and $b \in \mathbb{R}^{C_{\text{out}}}$. The layer produces patch embeddings $Y \in \mathbb{R}^{W' \times H' \times C_{\text{out}}}$.

A.1.2 Token Masking

The iBOT objective masks part of the image in the global views of the student network. Masking is done on the patch level.

For a given mask $M \in \{\text{true}, \text{false}\}^{W' \times H'}$ and patch embeddings $X \in \mathbb{R}^{W' \times H' \times C_{\text{out}}}$ the masked patches are substituted by a learnable mask embedding $t \in \mathbb{R}^{C_{\text{out}}}$

$$y_{i,j} = \begin{cases} t & \text{if } m_{i,j}, \\ x_{i,j} & \text{else.} \end{cases}$$

The mask token is initialized as $t = 0$.

A.1.3 Class and Register Tokens

DINOv3 adapts a ViT with registers [12]. Together with the [CLS] token a total of $P = \text{\#registers} + 1$ embeddings are prepended to the sequence. The patches $X \in \mathbb{R}^{W' \times H' \times C_{\text{out}}}$ are flattened into one dimension, and the feature dimension is permuted to be the first dimension. The prefix $P \in \mathbb{R}^{C_{\text{out}} \times P}$ is prepended, resulting in the sequence $Y \in \mathbb{R}^{C_{\text{out}} \times (P+W' \cdot H')}$

$$y_{k,l} = p_{k,l}$$

for $l \in \{1, \dots, P\}$ and $k \in \{1, \dots, C_{\text{out}}\}$, and

$$y_{k,P+(j-1) \cdot W' + i} = x_{i,j,k}$$

for $k \in \{1, \dots, C_{\text{out}}\}$, $j \in \{1, \dots, H'\}$ and $i \in \{1, \dots, W'\}$. The first prefix embedding p_1 corresponds to the [CLS] token.

A.1.4 Transformer Block

The token sequence passes through multiple transformer blocks. Each transformer block takes a sequence $X \in \mathbb{R}^{C_{\text{out}} \times (P+W' \cdot H')}$ as input and produces an output $Y \in \mathbb{R}^{C_{\text{out}} \times (P+W' \cdot H')}$ of the same dimensions. Every transformer block consists of essentially two parts, the MHSA block and a FFN block. Each block features the layer corresponding to its name, preceded by a layer norm and followed by a scale layer, with all three layers encapsulated by a residual skip-connection as seen in Figure 2.

A.1.4.1 Layer Norm

For a given input $X \in \mathbb{R}^{C_{\text{out}} \times N}$ the layer norm layer normalizes and scales each token and adds a bias

$$y_i = \frac{x_i - \mathbb{E}[x_i]}{\sqrt{\text{Var}[x_i] + \epsilon}} \odot \gamma + \beta,$$

with parameters $\gamma \in \mathbb{R}^{C_{\text{out}}}$ and $\beta \in \mathbb{R}^{C_{\text{out}}}$.

A.1.4.2 Multi-Head Self-Attention with Rotary Positional Embeddings

The ViT encoder uses MHSA. The input sequence $X \in \mathbb{R}^{C_{\text{out}} \times (P+W' \cdot H')}$ is projected into the query (Q), key (K), and value (V) sequence

$$q_i = W_Q \times x_i + b_Q, \quad k_i = W_K \times x_i + b_K, \quad v_i = W_V \times x_i + b_V$$

using parameters $\mathbf{W}_{Q,K,V} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{out}}}$ and $\mathbf{b}_{Q,K,V} \in \mathbb{R}^{C_{\text{out}}}$ respectively. The embedding dimension is divided among the attention heads

$$q'_{d,h,m} = q_{(d+((h-1) \cdot C_{\text{head}})),m}$$

for Q' , K' , and V' respectively, where $C_{\text{head}} = C_{\text{out}}/N_{\text{head}}$ are the features per head. RoPEs [14] are applied to the queries and keys. The modified queries Q'' and keys K'' are computed respectively as

$$\begin{pmatrix} q''_{d,h,m} \\ q''_{d+C_{\text{head}}/2,h,m} \end{pmatrix} = \begin{pmatrix} \cos 2\pi p_d(m)/\theta_d & -\sin 2\pi p_d(m)/\theta_d \\ \sin 2\pi p_d(m)/\theta_d & \cos 2\pi p_d(m)/\theta_d \end{pmatrix} \times \begin{pmatrix} q'_{d,h,m} \\ q'_{d+C_{\text{head}}/2,h,m} \end{pmatrix}$$

for $d \in \{1, \dots, C_{\text{head}}/2\}$, $h \in \{1, \dots, N_{\text{head}}\}$ and $m \in \{P+1, \dots, P+W' \cdot H'\}$ with periods $\Theta = \{\theta_i = 100^{2 \cdot (i-1)/(C_{\text{head}}/2)} \mid i \in \{1, \dots, C_{\text{head}}/4\}\}$. The first half of the pairs is rotated according to the horizontal token coordinate m_x , while the second half is rotated according to the vertical token coordinate m_y . Here m_x and m_y denote the normalized horizontal and vertical image-space coordinates (not array-space coordinates) of the m -th patch token, respectively

$$\begin{aligned} m_x &= 2 \cdot (\text{mod1}(m - P, W') - 0.5)/W' - 1 \\ m_y &= 2 \cdot ([(m - P)/W'] - 0.5)/H' - 1 \end{aligned}$$

and $p_d(m)$ is

$$p_d(m) = \begin{cases} m_x & \text{if } d \leq C_{\text{head}}/4 \\ m_y & \text{else.} \end{cases}$$

DINOv3 additionally implements coordinate jittering, shifting and scaling for $p_d(m)$. We implement this feature. We omit the description, as the ImageNet-1k configuration opts out of these features. After applying the positional embeddings, the usual SDPA is computed. Attention scores $\mathbf{A}$ are computed as

$$\begin{aligned} a_{s,l,h} &= \frac{1}{\sqrt{C_{\text{head}}}} \cdot \sum_{d=1}^{C_{\text{head}}} k''_{d,h,s} \cdot q''_{d,h,l} \\ \alpha_{s,l,h} &= \frac{e^{a_{s,l,h}}}{\sum_{i=1}^{P+W' \cdot H'} e^{a_{i,l,h}}} \end{aligned}$$

The value tokens are combined by computing a weighted average using the attention scores

$$z'_{e,h,l} = \sum_{s=1}^{P+W' \cdot H'} v'_{e,h,s} \cdot \alpha_{s,l,h}.$$

The outputs of the individual attention heads are concatenated

$$z_{(d+((h-1) \cdot C_{\text{head}})),m} = z'_{d,h,m}.$$

The final output is computed as a linear projection

$$\mathbf{y}_i = \mathbf{W}_{\text{out}} \times \mathbf{z}_i + \mathbf{b}_{\text{out}}$$

with parameters $\mathbf{W}_{\text{out}} \in \mathbb{R}^{C_{\text{out}} \times C_{\text{out}}}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^{C_{\text{out}}}$. The MHSA layer's output is again a sequence with the same dimensions as the input $\mathbf{Y} \in \mathbb{R}^{C_{\text{out}} \times (P+W' \cdot H')}$.

A.1.4.3 Multi-Layer Perceptron

The FFN is a simple MLP consisting of two linear layers with a GELU activation. Outputs are computed by passing through the linear layers and activation function

$$\mathbf{y}_i = \mathbf{W}_{\text{out}} \times \text{gelu}(\mathbf{W}_{\text{in}} \times \mathbf{x}_i + \mathbf{b}_{\text{in}}) + \mathbf{b}_{\text{out}}$$

with parameters $\mathbf{W}_{\text{in}} \in \mathbb{R}^{C \times C_{\text{out}}}$, $\mathbf{b}_{\text{in}} \in \mathbb{R}^C$, $\mathbf{W}_{\text{out}} \in \mathbb{R}^{C_{\text{out}} \times C}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^{C_{\text{out}}}$. C is usually defined via a ratio against C_{out} . ViT-L/16 uses a FFN-ratio of 4.

A.1.4.4 Scale

The layer scale is computed as

$$\mathbf{y}_i = \mathbf{x}_i \odot \boldsymbol{\gamma}$$

with parameter $\boldsymbol{\gamma} \in \mathbb{R}^{C_{\text{out}}}$.

A.1.4.5 Stochastic Depth

DINOv3 utilizes stochastic depth [31] while pre-training. For the probability p block outputs are masked (see Figure 2)

$$\mathbf{Y} = \frac{1}{1-p} \odot \mathbf{X} \odot \mathbf{m} \quad \text{with } \mathbf{m} \sim \text{Bernoulli}(1-p).$$

The blocks are dropped per observation. Not all observations in one batch run through the same blocks.

A.1.4.6 Residual Skip-Connection

Every transformer block includes two residual connections [32] (see Figure 2). The input $\mathbf{X}$ is added back to the output of the enclosed layers

$$\mathbf{Y} = \mathbf{X} + \text{layers}(\mathbf{X}).$$

A.2 Projection Heads with Bottleneck

The DINO and iBOT heads follow the same general architectural pattern. The head uses an MLP with an L_2 -normalized bottleneck as described in the original DINO paper [2]. An input $\mathbf{x}_i$ is passed through the MLP

$$\mathbf{z}_i = \mathbf{W}_{\text{bn}} \times \text{gelu}(\mathbf{W} \times \text{gelu}(\mathbf{W}_{\text{in}} \times \mathbf{x}_i + \mathbf{b}_{\text{in}}) + \mathbf{b}) + \mathbf{b}_{\text{bn}}$$

before being L_2 -normalized and projected onto K prototypes

$$y_i = W_{\text{out}} \times \frac{z_i}{\|z_i\|_2}$$

with parameters $W_{\text{in}} \in \mathbb{R}^{C \times C_{\text{in}}}$, $b_{\text{in}} \in \mathbb{R}^C$, $W \in \mathbb{R}^{C \times C}$, $b \in \mathbb{R}^C$, $W_{\text{bn}} \in \mathbb{R}^{C_{\text{bn}} \times C}$, $b_{\text{bn}} \in \mathbb{R}^{C_{\text{bn}}}$ and $W_{\text{out}} \in \mathbb{R}^{K \times C_{\text{bn}}}$.

B Implementation Details

The implementation of this thesis deviates from the PyTorch reference implementation in several ways. We do not implement every option available beyond core components and configurations used in this thesis. In addition, our implementation differs in three notable ways.

A function compiled via Reactant.jl assumes the same control-flow patterns that were presented at compile time [22]. This includes intermediate buffer sizes. The iBOT objective requires a random number of patches to be masked in the student. However Reactant.jl expects the iBOT head to be passed a constant-sized input. We pass all patch embeddings through the iBOT head and supply an inverted mask to the Sinkhorn-Knopp algorithm to mask everything but the intended tokens (Algorithm 2a masks with 0 and Algorithm 2b masks with $-\infty$ respectively). We ensure that no non-finite floats are introduced due to the masking. The inverted mask is passed to the objective function to further ensure only the student’s originally masked patches and the teacher’s corresponding patches are considered. The implementation is equivalent at the cost of consuming more memory as more tokens are passed through the iBOT head.

The reference implementation uses an improved version of stochastic depth [31]. Rather than computing and then masking the results, computations are dropped. The observations are shuffled in the batch dimension and only the first $(1 - \text{drop rate})$ fraction of the batch is actually passed through the layer [6]. This thesis uses the traditional implementation, at the expense of compute efficiency and memory usage.

DINOv3 computes the KoLeo regularization term over small batches of 16 observations, possibly located on different GPUs. DINOv2 computes the KoLeo regularization term over all observations located on one GPU without cross-communication. This implementation computes the term over the whole batch. Cross-communication is automatically facilitated by Reactant.jl.

C Data Augmentation

We follow the PyTorch reference implementation [1], [18] with a few adaptations. DINOv3 by default only performs horizontal flips. Because the orientation of the tiles is not semantically meaningful, we additionally allow vertical flips. Both flips are applied with a 50% probability. Global views contain between 32% and 100% of the original image at a resolution of 256×256. Local views range from 5% to 32% at a resolution of 128×128. Crops are taken with random side ratios between 3/4 and 4/3 and are resized and bilinearly interpolated to the given resolution.

We apply a color jitter with a probability of 80%, adjusting brightness (± 0.4), contrast (± 0.4), saturation (± 0.2), and hue ($\pm 36^\circ$) in a random order. We grayscale the image with a probability of 20%. Gaussian blurs are applied to the local view with a probability of 50%. The first global view is blurred. The second global view is blurred with a probability of 10% and solarized (threshold 0.5) with a probability of 20%. All Gaussian blurs use a kernel size of 9 and a random standard deviation between 0.1 and 2.0 along both axes. All views are normalized with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225).

We follow the official DINOv3 implementation on masking, which in turn follows the strategy of iBOT [7]. Masks for the iBOT objective are passed with the views and generated during data augmentation. Each global view receives a mask with a probability of 50%. Masks cover between 10% and 50% of the patches. Masks are produced by placing rectangles of random size (4 to 64 patches) and dimension ratios (between 0.3 and 3.3) until ten placement attempts have failed.

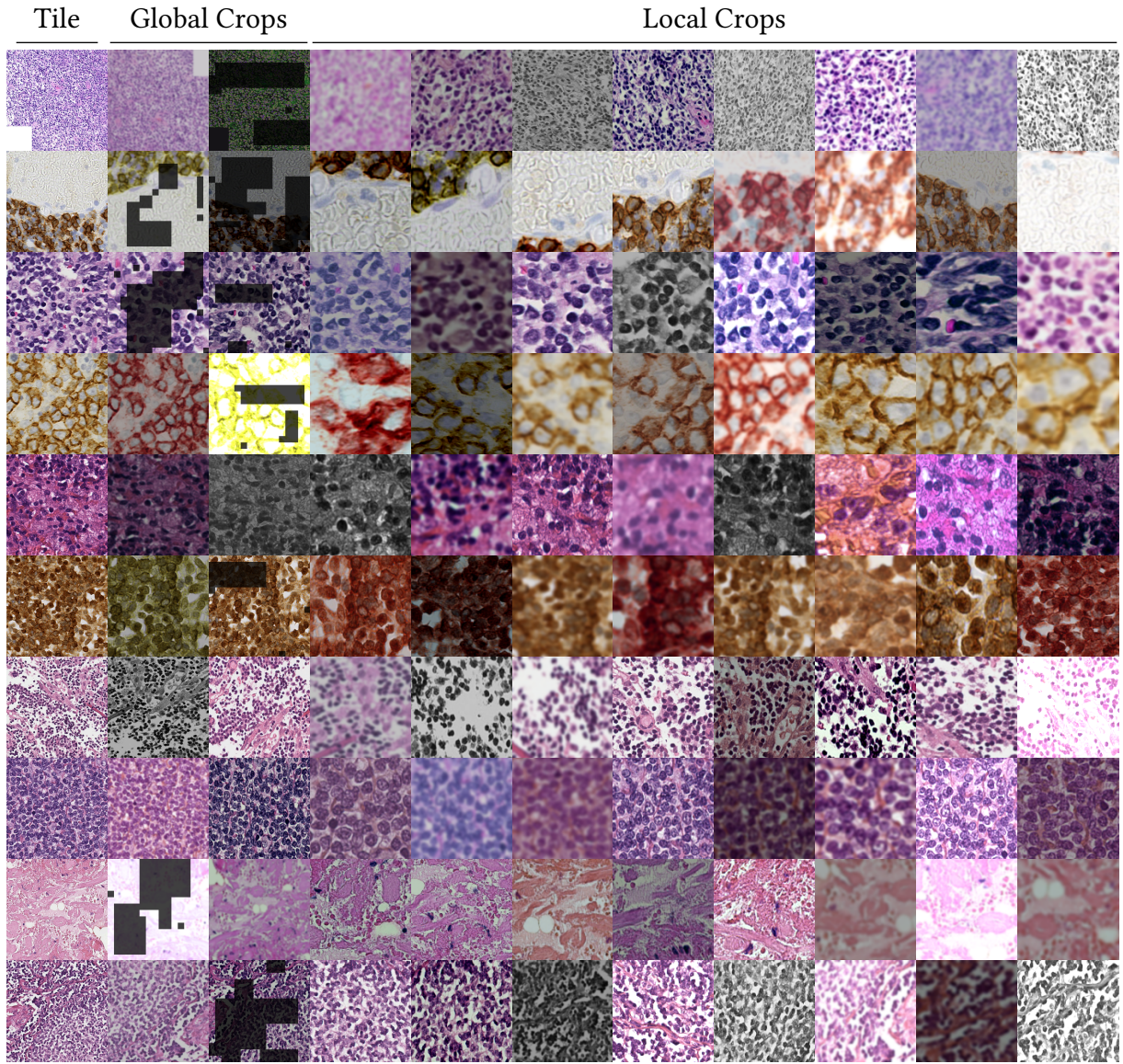


Figure 6: Samples of the Data Augmentation Views are generated from the original tile (left). Two global views (middle) and eight local views (right) are produced. Data augmentations include flipping, color jitter, grayscale, blur, and solarization. Masks are generated for approximately half of the global views. We depict them by overlaying the corresponding patches in black.

D Reproducibility

Lux.jl states include the current RNG state for all stochastic layers. To reproduce the training configuration used in this thesis, initialize the models using the scripts provided in the public repository.

The exact ordering of the training samples used in this thesis cannot be reproduced, as data loading on multiple distributed devices breaks deterministic ordering guarantees. In order to run reproducible data loading, consider sequential loading and passing an explicit RNG state. All data-augmentations accept an explicitly passed RNG state as well.

We use Julia v1.13.0. We run Enzyme v0.13.204, Lux v1.31.4, and Reactant v0.2.287. For a full overview of the package versions used, please refer to <https://github.com/benjwweber/BachelorThesis/Project.toml> and <https://github.com/benjwweber/BachelorThesis/Manifest.toml>.

We use four NVIDIA A100-SXM4-80GB with Compute Capability 8.0. The following versions are used: CUDA Driver: 13.0.0 [570.211.1]; CUDA Runtime: 13.1.0; CUDA Toolkit: 13.1.0; cuDNN: 9.21.0.

E Additional Implementations

This section provides an overview of evaluation methods implemented for this thesis, but ultimately not used. We do so to provide some context for the reference in Section 5.

E.1 Attention-based Multiple-Instance Learning

We implemented an adaptation of the AbMIL [33] model to evaluate the resulting ViT model. In general, a future model cannot be validated directly. Therefore a model’s performance is evaluated based on its viability as a feature extractor. An AbMIL classifier is trained on labeled slides and we compare the performance against a classifier trained using UNI’s features.

Multiple instance learning (MIL) operates on so-called bags of instances as inputs, in our case slides consisting of tiles in the latent space, and aims to assign a single class label to those bags.

Given a slide $S = \{t_i \in \mathbb{R}^{C_{\text{out}}} \mid i \in I\}$ consisting of the corresponding tiles at the maximum zoom level in latent space, the embedding dimension is further reduced by a linear layer with parameters $\mathbf{W}_{\text{in}} \in \mathbb{R}^{512 \times C_{\text{out}}}$ and $\mathbf{b}_{\text{in}} \in \mathbb{R}^{512}$,

$$\mathbf{h}_i = \text{relu}(\mathbf{W}_{\text{in}} \times \mathbf{t}_i + \mathbf{b}_{\text{in}})$$

The gated attention layer, as described in [33], produces attention weights for each tile in the slide, using three linear projections with parameters $\mathbf{W}_{\text{U}} \in \mathbb{R}^{256 \times 512}$, $\mathbf{b}_{\text{U}} \in \mathbb{R}^{256}$, $\mathbf{W}_{\text{V}} \in \mathbb{R}^{256 \times 512}$, $\mathbf{b}_{\text{V}} \in \mathbb{R}^{256}$ and $\mathbf{W}_{\text{w}} \in \mathbb{R}^{1 \times 256}$, $\mathbf{b}_{\text{w}} \in \mathbb{R}^1$

$$a_i = \mathbf{W}_{\text{w}} \times (\tanh(\mathbf{W}_{\text{U}} \times \mathbf{h}_i + \mathbf{b}_{\text{U}}) \odot \text{sigmoid}(\mathbf{W}_{\text{V}} \times \mathbf{h}_i + \mathbf{b}_{\text{V}})) + \mathbf{b}_{\text{w}}.$$

Here $\odot$ is the element-wise multiplication operator. The attention scores are normalized with a softmax so that they sum to 1

$$\alpha_i = \frac{e^{a_i}}{\sum_{j \in I} e^{a_j}}.$$

Tiles are reduced to one slide representative $\mathbf{s}$ by a weighted average of the tiles in the reduced latent space

$$\mathbf{s} = \sum_{i \in I} \alpha_i \cdot \mathbf{h}_i.$$

Finally the slide representative $\mathbf{s}$ is classified via a linear layer with parameters $\mathbf{W}_{\text{out}} \in \mathbb{R}^{n \times 512}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^n$ where n is the number of classes we want to predict

$$\mathbf{y} = \mathbf{W}_{\text{out}} \times \mathbf{s} + \mathbf{b}_{\text{out}}.$$

The output $\mathbf{y} \in \mathbb{R}^n$ can be softmaxed to produce class probabilities.

E.1.1 Batched Attention-based Multiple-Instance Learning

Reactant currently does not support compilation with dynamically sized arrays [22]. Inputs to the AbMIL model are therefore padded such that each slide contains the same number of tiles. A mask m_i identifies whether a tile t_i is part of the slide ($m_i = \text{true}$) or part of the padding ($m_i = \text{false}$). The scores of masked tiles are set to be minus infinity before applying softmax to ensure the padding does not influence the model’s output,

$$a_i = \begin{cases} \mathbf{W}_{\text{w}} \times (\tanh(\mathbf{W}_{\text{U}} \times \mathbf{h}_i + \mathbf{b}_{\text{U}}) \odot \text{sigmoid}(\mathbf{W}_{\text{V}} \times \mathbf{h}_i + \mathbf{b}_{\text{V}})) + \mathbf{b}_{\text{w}} & \text{if } m_i, \\ -\infty & \text{else.} \end{cases}$$

This prevents Reactant from having to recompile for differently sized slides and allows the compilation cache to be used effectively while training. Furthermore, this enables mini-batch learning with multiple batches at once, usually prevented by the nature of MIL’s differently sized input bags.